

Vulnerability Proof-of-Concepts in Software Supply Chain: Real-World Usage Scenarios from the Perspective of Migration

Wenjing Dang
Tianjin University
Tianjin, China
dangwenjing@tju.edu.cn

Xiaohong Li
Tianjin University
Tianjin, China
xiaohongli@tju.edu.cn

Sen Chen[✉]
Nankai University
Tianjin, China
senchen@nankai.edu.cn

Abstract

The security community widely uses software composition analysis (SCA) to identify vulnerabilities introduced by third-party libraries (TPLs) in the software supply chain. However, static analysis produces many false positives, so a Proof-of-Concept (PoC) is needed to confirm that a vulnerability is exploitable. Existing research studies PoC migration for cross-project vulnerability propagation. Propagation across operating systems and hardware architectures has received far less attention, yet each type raises distinct migration requirements and challenges. In this paper, we present three real-world usage scenarios of PoC migration, one for each propagation type: cross-project, cross-OS, and cross-architecture. For each scenario, we analyze its challenges and propose preliminary solutions by synthesizing existing research and empirically analyzing dozens of reproduced PoC cases. We regard the difference between the original and target vulnerability vectors as the mutation that a vulnerability undergoes as it propagates. Building on this view, we reuse an already-verified PoC_o as a high-fidelity seed and propose diff-based PoC migration as the preliminary solution. We further discuss how, in the era of large language models (LLMs), an agentic workflow can use LLMs to interpret these differences and to drive the program-analysis tools that adapt the PoC. We hope this work provides a basis for studying real-world PoC migration and for strengthening supply-chain security.

CCS Concepts

• Security and privacy → Software and application security.

Keywords

Software supply chain, Vulnerability propagation, Vulnerability validation, Proof-of-Concept, PoC

ACM Reference Format:

Wenjing Dang, Xiaohong Li, and Sen Chen. 2026. Vulnerability Proof-of-Concepts in Software Supply Chain: Real-World Usage Scenarios from the Perspective of Migration. In *Proceedings of the 2nd International Workshop on Software Genomics (SWGeno '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3844178.3844647>

1 Introduction

The software supply chain production model has improved the quality and efficiency of modern software development, but it has

also expanded the propagation of vulnerabilities [15]. Existing software composition analysis (SCA) tools detect vulnerable third-party library (TPL) calls through project library management files or code similarity analysis. This static approach only identifies the existence of the vulnerability code and cannot test the exploitability of the vulnerability, which brings a lot of false positives [6, 17].

A Proof-of-Concept (PoC) is demonstrative code or a report used to verify the existence and exploitability of a vulnerability. Due to differences in the vulnerability code context between the vulnerable TPL and its client projects, the original PoC for the TPL requires modifications to be applicable to the client projects. To address this problem, existing works have proposed PoC migration techniques for cross-project vulnerability propagation [6, 9, 10, 18].

However, vulnerability propagation in the software supply chain is multilayered and multifaceted. For example, the well-known Heartbleed vulnerability can potentially affect various software and systems such as web servers, Linux systems, and embedded devices. Existing research has explored PoC migration for cross-project vulnerability propagation (cross-project for short). In contrast, propagation across operating systems (cross-OS) and hardware architectures (cross-architecture) has received limited attention. The vulnerability vector difference (as defined in Section 3.1) varies by type: for cross-project it is the differing vulnerability-code context; for cross-OS it lies in system design, kernel customization, and package-manager patches; and for cross-architecture it lies in system calls, stack structure, and register layout. Accordingly, we propose three corresponding PoC migration scenarios.

PoC migration fits this setting well. The upstream project usually ships a PoC_o that is already verified to trigger the vulnerability. We can reuse it as a high-fidelity seed and only adapt it to the “mutation” (i.e., the *Diff*) that propagation introduces. This keeps vulnerability verification precise, efficient, and interpretable. Large language models (LLMs) offer further help. An LLM can summarize the *Diff* from diverse sources, such as documentation, patches, and change logs. Acting as the reasoning core of an agent, it can then drive program-analysis tools, such as directed fuzzers, debuggers, and constraint solvers, to adapt the PoC step by step.

This process also invites a reading through a software-genomics lens. The vulnerable code acts as a heritable gene, and each propagation edge imprints a *Diff*, that is, a mutation, upon it. A PoC is the assay that tests whether the vulnerability is still expressed after the mutation. PoC migration is then the problem of re-deriving this assay across the *Diff*. Guided by this view, we make three contributions. First, we systematize vulnerability propagation in the software supply chain into three types (cross-project, cross-OS, and cross-architecture) and cast each as a real-world PoC migration scenario. Second, we formally define the PoC migration problem,



This work is licensed under a Creative Commons Attribution 4.0 International License. *SWGeno '26, Munich, Germany*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3002-3/2026/10
<https://doi.org/10.1145/3844178.3844647>

and, through empirical studies that reproduce and analyze dozens of PoC cases, we characterize the *Diff* and migration challenges of each type. Third, we distill preliminary diff-based, LLM-assisted migration guidelines and outline a research agenda toward an agentic PoC-migration system.

2 Background

2.1 Vulnerability Propagation Types

2.1.1 Cross-project Propagation. Client projects invoke the TPL in two ways: by importing it through package management or by copying it from an open-source repository. Copy-based TPL calls can be further categorized into partial code copies and hard forks.

❶ **Package Manager-based TPL Calls.** Client projects interact directly with the TPL’s application programming interface (API) without modifying the TPL code logic. ❷ **Partial Code Copy.** Client projects add TPL source code to their own projects, and there may be customization changes. ❸ **Hard Fork.** Development may fork into parallel routes over philosophical or refactoring differences. For example (Figure 1), Neovim refactored Vim for extensibility [2]; they share substantial code, so vulnerabilities are likely to affect both [12], and bidirectional feature transfer further propagates them.

2.1.2 Cross-OS Propagation. ❹ **Linux Distribution Branching.** Linux distributions are built from upstream projects and primarily comprise the upstream kernel, the GNU project, and user-space software. Vendors assess upstream vulnerabilities against their distributions [11]; e.g., the Ubuntu CVE Tracker [5] analyzes dependencies and versions to scope impact, but this static approach still yields false positives.

2.1.3 Cross-architecture Propagation. ❺ **Multi-architecture Code Implementations.** With the rise of IoT, cross-architecture firmware and software vulnerabilities are increasingly common; e.g., Pewny et al. [14] detected Heartbleed in both MIPS-based router firmware and ARM-based NAS firmware. Since architectural differences change how a vulnerability triggers, PoC verification is necessary.

2.2 Vulnerability Proof-of-Concepts

A PoC is an important basis for reproducing vulnerabilities, typically a small piece of code plus a triggering step. Beyond verification, PoCs also underpin exploitability assessment and vulnerability migration [7, 10]. More recently, LLMs and LLM-driven agents have been used to automatically generate or reproduce PoCs and exploits from vulnerability information, such as CVE descriptions or binary analysis [8, 13]. This marks a shift toward agentic vulnerability analysis. Our work is complementary. Instead of synthesizing a PoC from scratch, we reuse an existing, verified PoC and migrate it across propagation boundaries, and we discuss how such agents can further serve the migration task.

3 PoC Migration Problems

3.1 Overall Problem Definition

A “vulnerability vector” (VV) denotes where a vulnerability resides in a given propagation type. For our three types, it is a software

project, a Linux distribution, or an instruction set architecture (ISA), respectively. Throughout the paper, subscripts o and t mark the original and target vectors: VV_o/VV_t are the vectors, VC_o/VC_t the vulnerability code they contain, V_o/V_t the resulting vulnerabilities, and PoC_o/PoC_t the corresponding PoCs, while *Diff* denotes the difference between VV_o and VV_t .

PoC Migration Problems: Consider an original vulnerability vector VV_o whose vulnerability code VC_o introduces a vulnerability V_o . It propagates to a target vector VV_t with corresponding code VC_t . Because of the differences (*Diff*) between VV_o and VV_t , it is unclear whether V_t in VV_t is exploitable as V_o is in VV_o , so its exploitability must be verified.

Let PoC_o be a proof of concept that triggers V_o . The problem is to migrate PoC_o , given the *Diff*, into a corresponding PoC_t that confirms the exploitability of V_t in VV_t .

3.2 Specific Problems Analysis

3.2.1 Cross-project PoC Migration Scenarios. ❶ **Package Manager-based TPL Calls.** The primary challenge is to identify a reachable path from the entry function of VV_t to the VC_o in VV_o . Additional complexity arises from the multi-level chain of TPL calls: the upstream library VV_o reaches downstream libraries $VV_{t1}, VV_{t2}, \dots$, so migration must iteratively generate $PoC_{t1}, PoC_{t2}, \dots$ ❷ **Partial Code Copy.** When reusing code, developers often make customized changes, so VC_t differs from VC_o in context location and syntactic structure. Hence PoC_t must both guide the control flow to reach VC_t and satisfy the constraints these modifications impose. ❸ **Hard Fork.** In hard fork projects, *Diff* contains both code logic and project configuration. As a result, PoC migration demands not only dynamic automated testing but also manual adjustments to software settings.

3.2.2 Cross-OS PoC Migration Scenarios. ❹ **Linux Distribution Branching.** An operating system’s composition is complex, yielding diverse *Diff*. By the degree of coupling with the OS, we categorize *Diff* into $Diff_{kernel}$ (Linux kernel), $Diff_{GNU}$ (GNU system utilities), and $Diff_{user_space}$ (user-space software), each requiring a different migration approach.

3.2.3 Cross-architecture PoC Migration Scenarios. ❺ **Multi-architecture Code Implementations.** Cross-architectural *Diff* involves the underlying computer design such as the instruction set, stack structure, and registers. There are high theoretical and technical challenges for analyzing vulnerabilities and debugging PoCs.

4 PoC Migration Guidelines

This section provides preliminary solutions for three PoC migration scenarios by summarizing existing work and conducting an empirical study. Our empirical process has three steps: we reproduce each PoC case to identify the necessary modifications, summarize the *Diff* from multi-source information, and map the modifications to the *Diff* to derive preliminary solutions.

4.1 Cross-project PoC Migration Guidelines

❶ **Package Manager-based TPL Calls.** Early work focused on passing the vulnerability function in VV_t the same parameters it

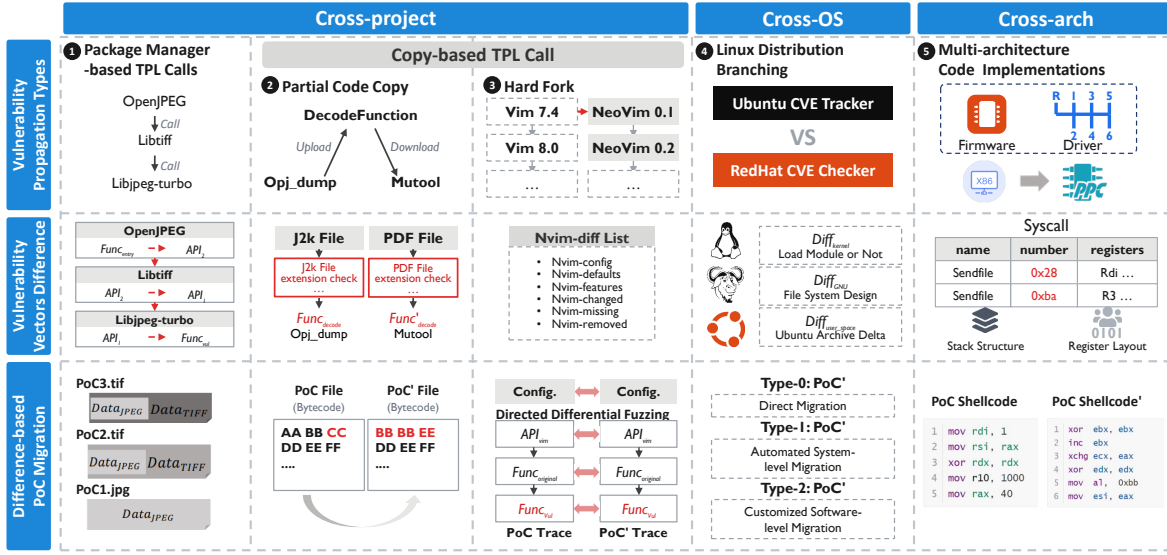


Figure 1: Real usage scenarios of vulnerability PoCs in the software supply chain.

receives at runtime in VV_o , thereby reproducing the vulnerability [6, 9]. Zhou et al. [18] discovered that accessing the vulnerability function from the entry point of VV_t involves traversing a multi-step function call chain, which constrained the success rate of PoC migration. To mitigate this, they applied step-by-step fuzzing along the function chain. Similarly, for multiple VV_t , we can use a divide-and-conquer strategy to incrementally generate PoC_t . **Partial Code Copy**. Regarding code reuse, the clone detection tool reports results as a discontinuous set of shared functions. OCTOPOCS [10] first identifies the bytes of PoC_o used by the shared function set as crash primitives. It then generates the remaining bytes of the PoC through constraint solving and fuzzing, forming bootstrap inputs that drive VV_t to execute VC_t . Finally, the bootstrap inputs and crash primitives are combined to form PoC_t of VV_t . **Hard Fork**. We reproduced 30 Vim PoCs in Neovim: 18 succeeded and 12 failed. Analyzing one failure (CVE-2022-2000), we found that Neovim’s `normal_check` function uses different branch-jumping logic than Vim, so PoC_o cannot trigger V_t . According to the official documentation, the *Diff* between Neovim and Vim consists of modifications to the default software configuration, the API interface, and the behavior of local functions. In practice, an LLM can automatically distill this *Diff* from documentation and change logs to guide the migration.

Based on these observations, we propose directed differential fuzzing [16]. For software configuration, we build a configuration-difference control table to synchronize the settings that PoC_o relies on. For program execution, since the traces of V_o and V_t should be similar, we first dynamically instrument VV_o to extract the trace T_{vim} when PoC_o triggers V_o . We then set Neovim’s VC_t as the fuzzing target, maximizing the similarity between the target trace T_{neovim} and T_{vim} .

Remark: The challenge of cross-project PoC migration lies in the complexities of program analysis, which an agentic workflow can carry out in three steps: (i) perceive the *Diff* and identify the function call chain from the entry function to the vulnerable function; (ii) plan guided inputs that satisfy the constraints of the execution path; and (iii) orchestrate data-flow tracking and directed fuzzing to deliver the crash primitives to the vulnerability location, iterating until V_t is triggered.

4.2 Cross-OS PoC Migration Guidelines

4 Linux Distribution Branching. In our experiment, we attempted to migrate 50 Ubuntu PoCs to Fedora. Of these, 21 could not be reproduced on the source Ubuntu itself and were discarded. Of the remaining 29, 17 reproduced directly on Fedora and 12 required modification. For example, CVE-2023-4911 needed a permission-command replacement and a `lib.so.6` path fix, and CVE-2019-12735 needed the Vim modeline step skipped. The *Diff* spans three levels: *Diff_{kernel}* (default configuration switches, resource contention, and module loading) [19]; *Diff_{GNU}* (system-design differences between Debian- and Red Hat-based systems, e.g., file-system structure [4] and permission management); and *Diff_{user_space}* (package-manager customization of the build process, configuration files, and interfaces) [1].

Remark: The challenge of cross-OS PoC migration lies in the diversity of environmental factors. An agent can perceive and classify the *Diff*, then select one of three strategies. **Type-0** is Direct Migration ($PoC_t = PoC_o$). **Type-1** is Automated System-level Migration: for *Diff_{GNU}* and *Diff_{user_space}*, an LLM applies conversion rules to adapt the PoC automatically. **Type-2** is Customized Software-level Migration: it adjusts the PoC according to the patch from the Linux distribution package manager.

4.3 Cross-architecture PoC Migration Guidelines

Multi-architecture Code Implementations. We reproduced 10 PoCs on x86-64 and PowerPC; 2 required migration due to calling-convention differences. For instance, one PoC uses the `sendfile` syscall [3], numbered `0x28` with arguments in `rDI/rsi/rdx/r10` on x86-64 but `0xba` with arguments in `r3/r4/r5/r6` on PowerPC-64, so its shellcode must be adapted. In summary, architectures differ mainly in system calls, stack structure, and register layout.

Building on this, we outline a preliminary solution. First, we establish a mapping between call numbers and call parameters based on the calling conventions of different architectures, converting $ShellCode_o$ into $ShellCode_t$ according to these rules. Next, we use an LLM to correct any residual syntax errors in $ShellCode_t$. Recent work shows that LLMs, when placed in a loop of analysis, generation, and verification, can automatically produce and repair working exploit code [13]. Finally, we replace $ShellCode_o$ with the modified $ShellCode_t$ to construct the new PoC_t .

Remark: The challenge of cross-architecture PoC migration lies in the rigor required for the shellcode, which an agentic workflow can handle in three steps: (i) establish mapping rules for assembly instructions and registers from the cross-architecture calling conventions; (ii) compute address offsets from the vulnerable code and stack structure; and (iii) assemble the instructions and operands into the shellcode of PoC_t , with an LLM repairing residual syntax errors.

5 Future Plans

Our long-term goal is to unify the three scenarios into an agentic PoC-migration system. In this system, an LLM-driven agent perceives the *Diff* (the “mutation” along a supply-chain propagation edge) from multi-source information, and then drives program-analysis tools to adapt a verified PoC_o into PoC_t .

Concretely, we plan three lines of work. For hard-fork projects, we will validate the differential fuzzing approach and extend it to more project pairs, such as MariaDB and MySQL. For Linux distribution branching, we will build a categorization method that decides which PoCs need migration; it will be grounded in large-scale reproduction over Ubuntu and Fedora resources and in expert-summarized $Diff_{GNU}$ and $Diff_{user_space}$. We will also derive a cross-OS scheme that spans the three migration strategies (Type-0/1/2). For multi-architecture implementations, we will provide a shellcode conversion tool.

6 Conclusion

We discuss real usage scenarios of vulnerability PoCs in the software supply chain from a migration perspective. For each scenario, we summarize its challenges and propose guidelines grounded in existing research and empirical studies. By reusing a verified PoC as a high-fidelity seed and adapting it to the propagation-induced *Diff*, migration makes vulnerability verification precise, efficient, and interpretable. Framed as an agentic, *Diff*-driven process, it offers a practical building block for software-genome analysis and supply-chain security governance.

Acknowledgments

Supported by Science Challenge Project, No. TZ2025002.

References

- [1] 2026. Index of /ubuntu. <http://archive.ubuntu.com/ubuntu/>. (Accessed on 07/15/2026).
- [2] 2026. Introduction · neovim/neovim Wiki · GitHub. <https://github.com/neovim/neovim/wiki/Introduction>. (Accessed on 07/15/2026).
- [3] 2026. Linux syscall table: latest, x86 64-bit, x86-64 ABI. <https://syscalls.mebeim.net/?table=x86/64/x64/latest>. (Accessed on 07/15/2026).
- [4] 2026. MultiarchSpec - Ubuntu Wiki. <https://wiki.ubuntu.com/MultiarchSpec>. (Accessed on 07/15/2026).
- [5] 2026. Ubuntu CVE Tracker in Launchpad. <https://launchpad.net/ubuntu-cve-tracker>. (Accessed on 07/15/2026).
- [6] Zirui Chen, Xing Hu, Xin Xia, Yi Gao, Tongtong Xu, David Lo, and Xiaohu Yang. 2024. Exploiting Library Vulnerability via Migration Based Automating Test Generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639583
- [7] Jiarun Dai, Yuan Zhang, Hailong Xu, Haiming Lyu, Zicheng Wu, Xinyu Xing, and Min Yang. 2021. Facilitating vulnerability assessment through poc migration. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3300–3317. doi:10.1145/3460120.3484594
- [8] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. 2024. LLM Agents can Autonomously Exploit One-day Vulnerabilities. *arXiv preprint arXiv:2404.08144* (2024). doi:10.48550/arXiv.2404.08144
- [9] Hong Jin Kang, Truong Giang Nguyen, Bach Le, Corina S Păsăreanu, and David Lo. 2022. Test mimicry to assess the exploitability of library vulnerabilities. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 276–288. doi:10.1145/3533767.3534398
- [10] Seongkyeong Kwon, Seunghoon Woo, Gangmo Seong, and Heejo Lee. 2021. OTOPOS: automatic verification of propagated vulnerable code using reformed proofs of concept. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 174–185. doi:10.1109/DSN48987.2021.00032
- [11] Jiahui Lin, Haoxiang Zhang, Bram Adams, and Ahmed E Hassan. 2023. Vulnerability management in linux distributions: An empirical study on debian and fedora. *Empirical Software Engineering* 28, 2 (2023), 47. doi:10.1007/s10664-022-10267-7
- [12] Shengyi Pan, You Wang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2024. Automating Zero-Shot Patch Porting for Hard Forks. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 363–375. doi:10.1145/3650212.3652134
- [13] Wanzong Peng, Lin Ye, Xuetao Du, Hongli Zhang, Dongyang Zhan, Yunting Zhang, Yicheng Guo, and Chen Zhang. 2025. PwnGPT: Automatic Exploit Generation Based on Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11481–11494. doi:10.18653/v1/2025.acl-long.562
- [14] Jannik Powny, Behrad Garmany, Robert Gawlik, Christian Rossow, and Thorsten Holz. 2015. Cross-architecture bug search in binary executables. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 709–724. doi:10.1109/SP.2015.49
- [15] Yulun Wu, Zeliang Yu, Ming Wen, Qiang Li, Deqing Zou, and Hai Jin. 2023. Understanding the threats of upstream vulnerabilities to downstream projects in the maven ecosystem. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1046–1058. doi:10.1109/ICSE48619.2023.00095
- [16] Songtao Yang, Yubo He, Kaixiang Chen, Zheyu Ma, Xiapu Luo, Yong Xie, Jianjun Chen, and Chao Zhang. 2023. 1dFuzz: Reproduce 1-Day Vulnerabilities with Directed Differential Fuzzing. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 867–879. doi:10.1145/3597926.3598102
- [17] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972. doi:10.1145/3611643.3616299
- [18] Zhuotong Zhou, Yongzhuo Yang, Susheng Wu, Yiheng Huang, Bihuan Chen, and Xin Peng. 2024. Magneto: A Step-Wise Approach to Exploit Vulnerabilities in Dependent Libraries via LLM-Empowered Directed Fuzzing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)*. Sacramento, California, USA. doi:10.1145/3691620.3695531
- [19] Xiaochen Zou, Yu Hao, Zheng Zhang, Juefei Pu, Weiteng Chen, and Zhiyun Qian. 2024. SyzBridge: Bridging the Gap in Exploitability Assessment of Linux Kernel Bugs in the Linux Ecosystem. In *31st Annual Network and Distributed System Security Symposium, NDSS*. doi:10.14722/ndss.2024.24926

Received 2026-07-27; accepted 2026-08-21